

Lab 4 Lock 实验报告

李睿阳 524120910059

1. Memory Allocator

实现思路

xv6 原生的 `kalloc` 机制中，所有 CPU 共用一个 `freelist` 和一把全局锁 `kmem.lock`。在高并发场景下，多个 CPU 同时申请或释放物理页会导致严重的锁竞争。

- **Per-CPU Freelists**: 将 `kmem` 结构由单体改为 `NCPU` 大小的数组。每个 CPU 维护自己的 `freelist` 和 `lock`。
- **本地优先分配**: `kalloc` 优先从当前 CPU 的链表中获取页面，不产生跨核冲突。
- **窃取机制 (Stealing)**: 当当前 CPU 链表为空时，遍历其他 CPU 的链表。如果发现其他 CPU 有空闲页面，则从中窃取一个。保证高并发性能，并且能够保证全局物理内存的利用率。

修改的文件

- [kernel/kalloc.c](#)

关键代码

```
struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];

void kinit() {
    for (uint64 i=0; i<NCPU; i++) {
        char name[8];
        snprintf(name, sizeof(name), "kmem_%d", i);
        initlock(&kmem[i].lock, name);
    }
    freerange(end, (void*)PHYSTOP);
}

void *kalloc(void) {
    struct run *r;
    push_off();
    int id = cpuid();
    acquire(&kmem[id].lock);
    r = kmem[id].freelist;
    if(r) {
        kmem[id].freelist = r->next;
    } else {
        for(int i = 0; i < NCPU; i++) {
            if(i == id) continue;
            acquire(&kmem[i].lock);
            if(kmem[i].freelist) {
                r = kmem[i].freelist;
                kmem[i].freelist = r->next;
                release(&kmem[i].lock);
                break;
            }
            release(&kmem[i].lock);
        }
    }
    release(&kmem[id].lock);
    pop_off();
    // .....
}
```

2. Buffer Cache

实现思路

原本的 `bcache` 使用单一 LRU 链表，所有的磁盘块访问都会竞争全局锁 `bcache.lock`。

- **哈希桶分段锁**: 用哈希表组织 Buffer。设置 `NBUCKETS = 13`，每个桶拥有独立的锁和哨兵节点 `head`。
- **移除全局 LRU**: 不维护 LRU 顺序。改为桶内 LRU 及遍历所有桶寻找空闲块。
- **两个阶段的查找与替换**:
 - i. **Fast Path**: 计算哈希值，获取桶锁，在桶内查找。如果命中，增加 `refcnt` 后直接返回。
 - ii. **Slow Path**: 如果未命中，获取全局 `bcache.lock`，遍历所有桶寻找 `refcnt == 0` 的 Buffer 进行替换，并将其从原桶迁移至新映射的桶。
- **Double Check**: 在获取全局锁寻找替换块后，再次检查目标块是否在此期间已被其他 CPU 加载到桶中。

修改的文件

- [kernel/bio.c](#)

关键代码

```
#define NBUCKETS 13
struct bucket {
    struct spinlock lock;
    struct buf head;
};

struct {
    struct spinlock lock;
    struct buf buf[NBUF];
    struct bucket buckets[NBUCKETS];
} bcache;

static struct buf* bget(uint dev, uint blockno) {
    int h = hash(blockno);
    acquire(&bcache.buckets[h].lock);

    for(b = bcache.buckets[h].head.next; b != &bcache.buckets[h].head; b = b->next){
        if(b->dev == dev && b->blockno == blockno){
            b->refcnt++;
            release(&bcache.buckets[h].lock);
            acquiresleep(&b->lock);
            return b;
        }
    }

    acquire(&bcache.lock);

    for(int i = 0; i < NBUCKETS; i++){
        int tid = (h + i) % NBUCKETS;
        if(tid != h) acquire(&bcache.buckets[tid].lock);
        for(b = bcache.buckets[tid].head.prev; b != &bcache.buckets[tid].head; b = b->prev){
            if(b->refcnt == 0){
                if(tid != h){
                    b->next->prev = b->prev;
                    b->prev->next = b->next;

                    b->next = bcache.buckets[h].head.next;
                    b->prev = &bcache.buckets[h].head;
                    bcache.buckets[h].head.next->prev = b;
                    bcache.buckets[h].head.next = b;
                }
            }
        }
    }
}
```

```

        b->dev = dev; b->blockno = blockno; b->refcnt = 1; b->valid = 0;

    }
}
if(tid != h) release(&bcache.buckets[tid].lock);
}
}

```

3. 调试过程中遇到的问题

3.1 CPUID的安全性 (kalloc)

在 `kalloc` 和 `kfree` 中，必须通过 `push_off()` 关闭中断后再调用 `cpuid()`。

- 如果不关中断，进程可能在获取 CPU ID 之后被抢占。当它恢复执行时，可能已经被调度到另一个 CPU，导致持有了 CPU A 的锁却在操作 CPU B 的链表，导致系统崩溃或竞态。

3.2 锁的层级与死锁 (bio)

在 `bget` 的替换逻辑中，需要同时持有全局锁和桶锁：

- 必须先获取全局锁 `bcache.lock`，再获取具体的桶锁。
- 当从一个桶窃取 Buffer 移动到另一个桶时，需要持有两个桶锁。必须保证加锁顺序或在持有全局锁的前提下穿行处理，防止两个核互相等待对方持有桶锁的情况。

3.3 替换时的原子性与可见性 (bio)

- **漏掉二次检查**: 在 `bget` 获取全局锁后，必须再次检查。因为在释放当前桶锁到获取全局锁的间隙，可能有其他 CPU 已经处理了该块缺失。
- **引用计数**: `brelease` 释放时，一定要使用该 Buffer 当前 `blockno` 对应的哈希桶锁。